

PROGRAMMING 622 ASSIGNMENT



Name and Surname:	Thabo Modisadife		
Student ITS No:	402502611		
Qualification:	BSc IT	Year of Study:	2 nd
		Semester:	2 nd
Assignment due date:	9/21/2026	Date submitted:	9/21/2026

ASSIGNMENT INSTRUCTIONS

Please tick each box to confirm completion.

Use Times New Roman font, size 12, with 1.5 line spacing throughout the document. Apply Harvard Referencing Style for all citations and references.

For essay-style assignments, please include the following sections:

Table of Contents Introduction

Main Body (with relevant subheadings) Conclusion

References

Submit the assignment in PDF format on Moodle. Use the specified cover page provided.

Include a signed declaration of originality.

DECLARATION OF ORIGINALITY:

I hereby declare that this assignment is my own work and has not been copied from any other source except where due acknowledgment is made. I affirm that all sources used have been properly cited and that this submission complies with the Institution's policies on academic integrity and plagiarism.

Student Signature: _____

Date: _____

9/21/2026

Contents

.....	1
Linear Search vs Binary Search	4
Insertion and Traversal Complexities comparison.....	5
Traversal Insertion:	5
Insertion.....	6
Why Stack and Queue is appropriate	8
Stack.....	8
Queue.....	8
Bibliography	9

Figure 1: Linear vs Binary search.....	4
Figure 2: Stack vs Queue	8

Linear Search vs Binary Search

Linear search, also known as sequential search is the simplest search algorithm. It operates by sequentially checking each element in a list until a match is found or the entire list has been traversed. It is an algorithm that is applicable when data is unsorted or when dataset is small, making the overhead of sorting less efficient than a direct linear scan (Fiha, 2026).

Binary search is a more efficient algorithm for finding an element within a list, but it comes with a crucial prerequisite: the data must be sorted. Binary search does not check every element like linear search, instead it employs a “divide and conquer” strategy. It repeatedly divides the search interval in half (Fiha, 2026).

Linear search Vs Binary Search

Find "J"

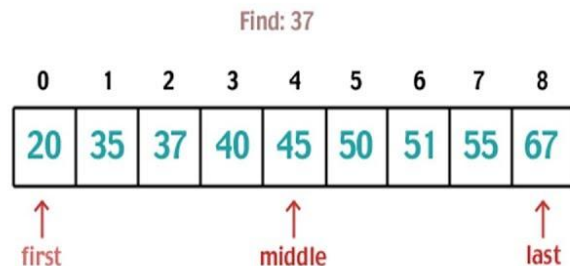
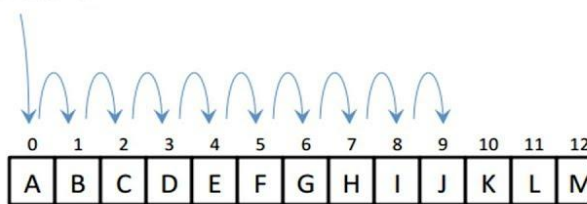


Figure 1: Linear vs Binary search

Insertion and Traversal Complexities comparison

Insertion and Traversal are part of Arrays, which are fundamental data structure in programming that allow you to store multiple elements of the same type in a contiguous block of memory.

Traversal Insertion:

It involves accessing each element of an array exactly once to perform an operation, such as printing the elements or calculating the sum (Sah, 2024).

Example (Sah, 2024):

```
#include <iostream>
using namespace std;

void traverseArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int size = sizeof(arr) / sizeof(arr[0]);
    traverseArray(arr, size);
    return 0;
}
```

The best case for its complexity is $O(n)$, average case is $O(n)$, and the worst case is $O(n)$

Insertion

Insertion involves adding an element to the array. If the array is not full this operation can be straightforward, but if the array is full, it may require creating a new array with a larger size (Sah, 2024).

Example (Sah, 2024):

```
#include <iostream>
using namespace std;

void insertElement(int arr[], int &size, int element, int position) {
    if (position > size) {
        cout << "Insertion not possible. Position out of bounds." << endl;
        return;
    }

    for (int i = size; i > position; i--) {
        arr[i] = arr[i - 1];
    }

    arr[position] = element;
    size++;
}

int main() {
    int arr[10] = {1, 2, 3, 4, 5};
    int size = 5;
    insertElement(arr, size, 10, 2);
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
}
```

Thabo Modisadife 402502611

```
    return 0;  
}
```

The best case($O(1)$ insertion at the end) , average case($O(n)$), worst case ($O(n)$ insertion at the beginning (Sah, 2024).

Why Stack and Queue is appropriate

Stack and Queue are linear data structures used to store and manage data efficiently (GeeksforGeeks, 2026).

The Stack follows the LIFO (Last in, First Out) principle (GeeksforGeeks, 2026).

The Queue follows the FIFO (First in, First Out) principle (GeeksforGeeks, 2026).

Stack

Follows a LIFO meaning that the last element added to the stack is the first one to be removed. It can be visualized as a pile of plates where you can only add or remove the top plate (GeeksforGeeks, 2026).

The operations relevant to the NeoVerse AI project:

- Push: Adds an element to the top of the stack.
- Pop: Removes and returns the top element of the stack.

Queue

It can be visualized as a line of people waiting for a service, where the first person in line is the first to be served (GeeksforGeeks, 2026).

The operations relevant to the NeoVerse AI project:

- Enqueue: Adds an element to the end (rear) of the queue (GeeksforGeeks, 2026).
- Dequeue: Removes and returns the front element of the queue (GeeksforGeeks, 2026).

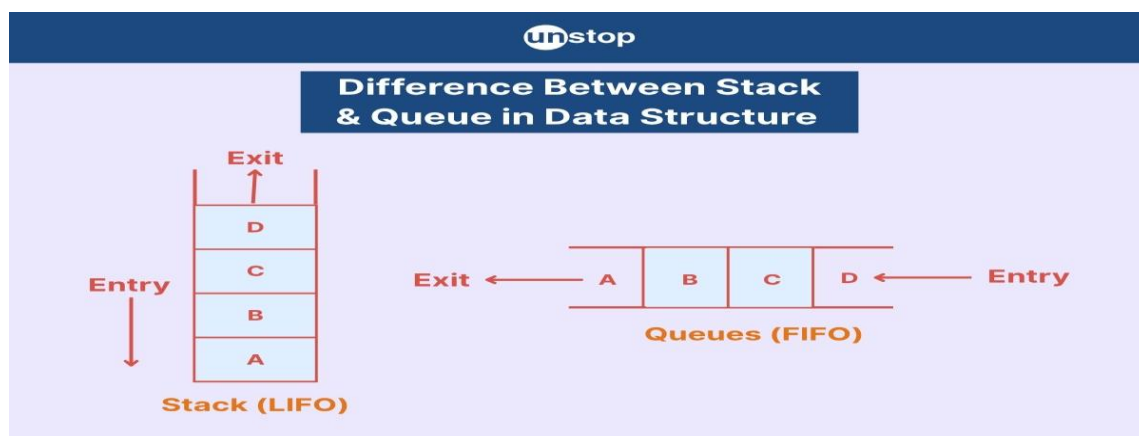


Figure 2: Stack vs Queue

Bibliography

Fiha, G., 2026. *Medium*. [Online]

Available at: <https://medium.com/@gulsaba.fiha/linear-search-vs-binary-search-in-algorithm-4c5bab9811dd>

[Accessed 19 September 2026].

GeeksforGeeks, 2026. *GeeksforGeeks*. [Online]

Available at: <https://www.geeksforgeeks.org/dsa/difference-between-stack-and-queue-data-structures/>

[Accessed 19 September 2026].

Sah, G., 2024. *Medium*. [Online]

Available at: <https://gauravssah.medium.com/operations-on-arrays-in-data-structures-traversal-insertion-deletion-and-searching-d90b7ec5c244>

[Accessed 19 September 2026].